문서 전처리 가이드 개요

이 문서는 RAG(Retrieval-Augmented Generation) 시스템, Knowledge Graph, Data Mesh/Fabric을 위한 문서 전처리 파이프라인 구축 가이드입니다.

문서 구성

```
docs/preprocessing/
├── 00_OVERVIEW.md                      # 전체 개요 (현재 문서)
├── 01_RAG_PREPROCESSING.md             # RAG를 위한 전처리
├── 02_KNOWLEDGE_GRAPH_PREPROCESSING.md # Knowledge Graph 전처리
├── 03_DATA_MESH_FABRIC_PREPROCESSING.md# Data Mesh/Fabric 전처리
├── 04_CHUNKING_STRATEGIES.md           # 청킹 전략 상세 가이드
└── 05_EVALUATION_BENCHMARKING.md       # 평가 및 벤치마킹
```

빠른 시작 가이드

목적별 문서 선택

목적	권장 문서
RAG 시스템 구축	01_RAG_PREPROCESSING.md
지식 그래프 구축	02_KNOWLEDGE_GRAPH_PREPROCESSING.md
엔터프라이즈 데이터 플랫폼	03_DATA_MESH_FABRIC_PREPROCESSING.md
청킹 전략 최적화	04_CHUNKING_STRATEGIES.md
품질 평가 및 모니터링	05_EVALUATION_BENCHMARKING.md

경험 수준별 가이드

입문자:

1. 현재 문서(00_OVERVIEW.md)로 전체 구조 이해
2. 01_RAG_PREPROCESSING.md의 기본 파이프라인 학습
3. 04_CHUNKING_STRATEGIES.md의 기본 청킹 전략 적용

중급자:

1. 용도에 맞는 전처리 문서 선택 (01~03)
2. 04_CHUNKING_STRATEGIES.md에서 고급 청킹 전략 학습
3. 05_EVALUATION_BENCHMARKING.md로 품질 평가 적용

고급자:

1. 모든 문서의 고급 섹션 참조
2. 커스텀 파이프라인 구축
3. A/B 테스트 및 지속적 최적화

전처리 파이프라인 아키텍처

```
┌─────────────────────────────────────────────────────────────────────────┐
│                          문서 수집 (Document Ingestion)                  │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│   │   PDF    │  │   DOCX   │  │   HTML   │  │   HWP    │  │   기타   │  │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
│        └──────────┴──────────┴──────────┴──────────┴──────────┘         │
└─────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌─────────────────────────────────────────────────────────────────────────┐
│                            텍스트 추출 (Extraction)                      │
│   ┌─────────────────────────────────────────────────────────────────┐   │
│   │  Parser Selection → Content Extraction → Structure Parsing      │   │
│   └─────────────────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌─────────────────────────────────────────────────────────────────────────┐
│                             전처리 (Preprocessing)                       │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│   │ 정규화   |→| 정제     |→| 필터링   |→| 토큰화   |→| 메타데이터|  │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
└─────────────────────────────────────────────────────────────────────────┘
                                    │
                        ┌───────────┴───────────┐
                        │                       │
                        ▼                       ▼                       ▼
┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────┐
│    RAG Pipeline      │  │   Knowledge Graph    │  │   Data Mesh/Fabric   │
├──────────────────────┤  ├──────────────────────┤  ├──────────────────────┤
│ • 청킹               │  │ • 엔티티 추출        │  │ • 데이터 계보        │
│ • 임베딩             │  │ • 관계 추출          │  │ • 메타데이터 관리    │
│ • 인덱싱             │  │ • 트리플 생성        │  │ • 품질 검증          │
│ • 검색 최적화        │  │ • 그래프 구축        │  │ • 도메인 관리        │
└──────────────────────┘  └──────────────────────┘  └──────────────────────┘
          │                       │                       │
          └───────────────────────┴───────────────────────┘
                                    │
                                    ▼
┌─────────────────────────────────────────────────────────────────────────┐
│                          평가 및 모니터링 (Evaluation)                   │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│   | 품질평가 | | 벤치마킹 | | A/B테스트| | 모니터링 | | 알림     |  |
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
└─────────────────────────────────────────────────────────────────────────┘
```

문서별 상세 내용

01. RAG 전처리

RAG(Retrieval-Augmented Generation) 시스템을 위한 문서 전처리 파이프라인 구축 가이드입니다.

주요 내용:

- 전처리 파이프라인 아키텍처
- 텍스트 정규화 및 정제
- 다양한 청킹 전략 (고정, 의미적, 재귀적)
- 임베딩 생성 및 최적화
- 벡터 데이터베이스 연동 (Faiss, Pinecone, Milvus)
- 메타데이터 강화
- 하이브리드 검색 최적화
- 성능 최적화 기법

코드 예시:

```
from src.preprocessing import TextPreprocessor, SemanticChunker

preprocessor = TextPreprocessor(
    normalize_unicode=True,
    remove_extra_whitespace=True,
    fix_encoding=True
)

chunker = SemanticChunker(
    embedding_model="sentence-transformers/all-MiniLM-L6-v2",
    similarity_threshold=0.7,
    max_chunk_size=500
)

# 파이프라인 실행
text = preprocessor.process(raw_text)
chunks = chunker.chunk(text)
```

---

02. Knowledge Graph 전처리

지식 그래프 구축을 위한 엔티티/관계 추출 및 트리플 생성 가이드입니다.

주요 내용:

- Knowledge Graph 아키텍처
- 엔티티 인식 및 추출 (NER)
- 관계 추출 (Relation Extraction)

- 트리플 생성 및 검증
- 온톨로지 매핑
- 엔티티 해소 (Entity Resolution)
- 그래프 데이터베이스 연동 (Neo4j)
- GraphRAG 통합

코드 예시:

```
from src.knowledge_graph import EntityExtractor, RelationExtractor,
TripleGenerator

# 엔티티 추출
extractor = EntityExtractor(model="ko-ner")
entities = extractor.extract(text)

# 관계 추출
relation_extractor = RelationExtractor()
relations = relation_extractor.extract(text, entities)

# 트리플 생성
generator = TripleGenerator(ontology=ontology)
triples = generator.generate(entities, relations)
```

---

03. Data Mesh/Fabric 전처리

엔터프라이즈 데이터 플랫폼을 위한 분산 데이터 관리 가이드입니다.

주요 내용:

- Data Mesh 원칙 및 아키텍처
- Data Fabric 통합 패턴
- 데이터 계보(Lineage) 추적
- 메타데이터 관리
- 데이터 품질 검증
- 도메인 기반 데이터 관리
- 셀프 서비스 데이터 플랫폼
- 거버넌스 및 규정 준수

코드 예시:

```
from src.data_fabric import DataProduct, LineageTracker, QualityValidator

# 데이터 프로덕트 생성
product = DataProduct(
    name="customer_documents",
    domain="customer_service",
    owner="team_a"
)
```

```
# 계보 추적
tracker = LineageTracker()
tracker.record_transformation(source, target, operation)

# 품질 검증
validator = QualityValidator(rules=quality_rules)
report = validator.validate(data)
```

---

04. 청킹 전략

다양한 문서 유형과 용도에 맞는 청킹 전략 상세 가이드입니다.

주요 내용:

- 청킹 기본 원리 및 중요성
- 고정 크기 청킹
- 의미적(Semantic) 청킹
- 재귀적 청킹
- 문서 구조 기반 청킹
- 에이전틱(Agentic) 청킹
- 하이브리드 전략
- 청킹 최적화 및 평가

청킹 전략 비교:

전략	장점	단점	적합한 용도
고정 크기	빠름, 예측 가능	문맥 손실	대용량 배치 처리
의미적	문맥 보존	느림, 비용	정확도 중요한 QA
재귀적	구조 보존	설정 복잡	구조화된 문서
에이전틱	최고 품질	가장 느림	핵심 문서 처리

---

05. 평가 및 벤치마킹

전처리 품질 평가, 성능 측정, 모니터링 가이드입니다.

주요 내용:

- 평가 프레임워크 설계
- 청킹 품질 평가
- 임베딩 품질 평가
- 검색 성능 메트릭 (MRR, MAP, NDCG)
- 생성 품질 평가 (ROUGE, BLEU, BERTScore)
- End-to-End RAG 평가 (RAGAS)
- Knowledge Graph 평가
- 성능 벤치마킹

- A/B 테스트
- 모니터링 및 알림

평가 메트릭 요약:

평가 영역	주요 메트릭
청킹 품질	의미적 일관성, 경계 품질, 정보 보존율
임베딩 품질	의미 보존율, 클러스터링 품질, 실루엣 점수
검색 성능	Precision@K, Recall@K, MRR, MAP, NDCG
생성 품질	ROUGE, BLEU, BERTScore, Faithfulness
End-to-End	Answer Relevance, Context Precision/Recall

핵심 개념 용어집

용어	설명
RAG	Retrieval-Augmented Generation. 검색과 생성을 결합한 AI 기술
Chunking	긴 문서를 작은 단위로 분할하는 과정
Embedding	텍스트를 고차원 벡터로 변환한 표현
Vector Database	벡터 임베딩을 저장하고 유사도 검색을 수행하는 DB
Knowledge Graph	엔티티와 관계로 구성된 지식 표현 구조
Triple	(주어, 술어, 목적어) 형태의 지식 그래프 기본 단위
Entity Resolution	동일 엔티티의 다른 표현을 통합하는 과정
Data Mesh	도메인 중심의 분산 데이터 아키텍처
Data Fabric	통합 메타데이터 기반 데이터 관리 아키텍처
Data Lineage	데이터의 원본부터 현재까지의 변환 이력
MRR	Mean Reciprocal Rank. 첫 정답 순위의 역수 평균
NDCG	Normalized Discounted Cumulative Gain. 순위 품질 메트릭
RAGAS	RAG Assessment. RAG 시스템 평가 프레임워크
Faithfulness	생성된 답변이 컨텍스트에 충실한 정도

의존성 및 설치

필수 패키지

```
# 기본 전처리
pip install numpy pandas regex tqdm
```

```
# 텍스트 처리
pip install nltk spacy konlpy

# 임베딩 및 벡터 DB
pip install sentence-transformers faiss-cpu chromadb

# NLP/ML
pip install transformers torch scikit-learn

# 평가
pip install rouge-score bert-score sacrebleu evaluate

# Knowledge Graph
pip install networkx neo4j rdflib

# 모니터링
pip install prometheus-client psutil
```

선택적 패키지

```
# GPU 지원 (CUDA)
pip install faiss-gpu torch-cuda

# 고급 임베딩
pip install openai cohere

# 분산 처리
pip install ray dask

# 시각화
pip install matplotlib seaborn plotly
```

사용 사례별 가이드

1. 기업 문서 검색 시스템

```
권장 경로:
01_RAG_PREPROCESSING.md → 04_CHUNKING_STRATEGIES.md →
05_EVALUATION_BENCHMARKING.md

핵심 포인트:
- 다양한 문서 형식 지원 (PDF, DOCX, HWP)
- 한국어 특화 전처리
- 메타데이터 기반 필터링
- 하이브리드 검색 (키워드 + 의미)
```

2. 지식 베이스 구축

```
권장 경로:
02_KNOWLEDGE_GRAPH_PREPROCESSING.md → 01_RAG_PREPROCESSING.md →
05_EVALUATION_BENCHMARKING.md

핵심 포인트:
- 엔티티 및 관계 추출
- 트리플 생성 및 검증
- GraphRAG 통합
- 지식 그래프 품질 평가
```

3. 데이터 플랫폼 구축

```
권장 경로:
03_DATA_MESH_FABRIC_PREPROCESSING.md → 05_EVALUATION_BENCHMARKING.md

핵심 포인트:
- 도메인 기반 데이터 관리
- 데이터 계보 추적
- 품질 검증 자동화
- 거버넌스 구현
```

4. 고정밀 QA 시스템

```
권장 경로:
04_CHUNKING_STRATEGIES.md → 01_RAG_PREPROCESSING.md →
05_EVALUATION_BENCHMARKING.md

핵심 포인트:
- 의미적/에이전틱 청킹
- 컨텍스트 보존 최대화
- RAGAS 기반 평가
- A/B 테스트 지속 최적화
```

성능 최적화 체크리스트

전처리 단계

- ☐ 문서 형식별 최적 파서 선택
- ☐ 배치 처리 크기 최적화
- ☐ 병렬 처리 활성화
- ☐ 캐싱 전략 적용

청킹 단계

- ☐ 용도에 맞는 청킹 전략 선택
- ☐ 청크 크기 최적화 (보통 200-500 토큰)
- ☐ 오버랩 비율 조정 (10-20%)
- ☐ 메타데이터 보존

임베딩 단계

- ☐ 적절한 임베딩 모델 선택
- ☐ GPU 가속 활용
- ☐ 배치 임베딩 적용
- ☐ 임베딩 캐싱

인덱싱 단계

- ☐ 벡터 DB 선택 및 최적화
- ☐ 인덱스 파라미터 튜닝
- ☐ 샤딩/파티셔닝 전략
- ☐ 정기적 재인덱싱

평가 단계

- ☐ 평가 데이터셋 구축
- ☐ 자동화된 평가 파이프라인
- ☐ A/B 테스트 프레임워크
- ☐ 모니터링 대시보드

트러블슈팅

일반적인 문제

문제	원인	해결책
검색 정확도 낮음	청킹 품질	의미적 청킹 적용, 청크 크기 조정
메모리 부족	대용량 문서	스트리밍 처리, 배치 크기 축소
처리 속도 느림	직렬 처리	병렬 처리, GPU 활용
인코딩 오류	문자셋 불일치	UTF-8 강제 변환, 인코딩 감지
임베딩 품질 낮음	부적절한 모델	도메인 특화 모델 사용

로그 및 디버깅

```
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
```

```
        logging.FileHandler('preprocessing.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger('preprocessing')
```

참고 자료

공식 문서

- LangChain Documentation
- LlamaIndex Documentation
- Sentence Transformers
- RAGAS Documentation

논문

- "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"
- "Dense Passage Retrieval for Open-Domain Question Answering"
- "RAGAS: Automated Evaluation of Retrieval Augmented Generation"

커뮤니티

- LangChain Discord
- LlamaIndex Discord
- Hugging Face Forums

업데이트 이력

버전	날짜	변경 내용
1.0.0	2024-01	초기 문서 작성
1.1.0	2024-03	청킹 전략 문서 추가
1.2.0	2024-06	평가/벤치마킹 문서 추가
1.3.0	2024-09	Knowledge Graph 섹션 확장
1.4.0	2024-12	전체 문서 확장 및 코드 예시 추가

문의 및 기여

이 문서에 대한 피드백이나 개선 제안은 언제든 환영합니다.

- 이슈 리포트: GitHub Issues
- 기여 가이드: CONTRIBUTING.md

이 문서는 지속적으로 업데이트됩니다.